LAB3A: Writing an AWS EC2 Terraform Module

Step 1: Define the Purpose of the Module

- This module will:
- Create an AWS EC2 instance
- Allow reusable deployments
- Accept customizable inputs
- Output important resource information

Step 2 : Create the Module Directory Structure

```
terraform-project/
│
├── main.tf
├── provider.tf
├── variables.tf
├── terraform.tfvars
│
└── modules/
    └── ec2/
        ├── main.tf
        ├── variables.tf
        ├── outputs.tf
        └── README.md
```

Step 3: Write the EC2 Resource (modules/ec2/main.tf)

```
resource "aws_instance" "ec2_instance" {
        ami                    = var.ami_id
         instance_type         = var.instance_type
        subnet_id              = var.subnet_id
        key_name               = var.key_name
       vpc_security_group_ids = var.security_groups

        tags = {
       Name        = var.instance_name
        Environment = var.environment
        }
}
```

Step 4: Define Input Variables (modules/ec2/variables.tf)

```
variable "ami_id" {
 description = "AMI ID for EC2 instance"
 type        = string
}

variable "instance_type" {
 description = "EC2 instance type"
type        = string
}
```

Step 5 : Add Local Values (Optional Enhancement)

```
locals {
        common_tags = {
       ManagedBy = "Terraform"
        Project   = "DevOps-Lab"
         }
}
```

Step 6 : Define Outputs (modules/ec2/outputs.tf)

```
output "instance_id" {
  value = aws_instance.ec2_instance.id
}

output "public_ip" {
  value = aws_instance.ec2_instance.public_ip
}

output "private_ip" {
  value = aws_instance.ec2_instance.private_ip
}
```

Step 7: Call the Module from Root Configuration (main.tf)

```
module "ec2" {
  source = "./modules/ec2"

  ami_id          = "ami-0c55b159cbfafe1f0"
  instance_type   = "t2.micro"
  subnet_id       = "subnet-12345678"
  key_name        = "my-key"
  security_groups = ["sg-12345678"]

  instance_name = "web-server"
  environment   = "production"
}
```

Publishing a Terraform Module to the Terraform Registry

- Create the Terraform module
- Organize the module files properly (main.tf, variables.tf, outputs.tf, README.md)
- Test the module locally
- Create a Git repository for the module
- Push the module code to GitHub
- Follow Terraform Registry naming conventions
- Add proper module documentation in README.md

Publishing a Terraform Module to the Terraform Registry

- Create a semantic version tag (e.g., v1.0.0)
- Push the version tag to GitHub
- Sign in to the Terraform Registry
- Connect the GitHub account to the Terraform Registry
- Select the repository containing the module
- Publish the module to the Terraform Registry
- Verify the published module information and documentation
- Maintain and update the module using new version tags